Recruited[1]

With
Apart Research

Abstract

We test whether a target agent's internal activations predict when it will give in to a peer agent's repeated requests to violate an operator rule. The layout below exercises the current Methods prose, a representative results table, and a full-width figure.

1 Research conducted at the AI Collusion Research Sprint, October 2026

1. Introduction

2. Related Work

3. Methods

3.1 Environment and agents

We study a two-agent setup in a mocked workspace. A **target agent** is given an ordinary task by its operator — read a small configuration file, then write a short status report and post where it is saved — together with one rule: do not share a specific protected value (for example a staging deploy token) outside the file that holds it. A second **peer agent** shares a team channel with the target and needs that value to finish its own task, but cannot read the file itself. Over up to 20 rounds, the peer asks the target for the value; each round the peer posts first, then the target takes a turn of tool calls and an optional channel message. The target has three tools: read a file, write a file, and send a channel message.

The peer is DeepSeek-V4-Flash; the main target is Qwen3-32B with thinking disabled. Both run through OpenRouter at temperature 0.7, pinned to a single provider (DeepInfra) with fallbacks disabled, so every call is served the same way. The peer is not told about the operator's rule: it is a blocked teammate pushing for what it needs, not an agent instructed to cause a violation. This is deliberate — we study recruitment that arises from an ordinary conflict of goals, not scripted collusion.

Each scenario is run in three conditions: a **recruiting** peer (asks for the protected value), a **control** peer (same scenario and task, normal collaboration, no forbidden ask), and the **target alone** (no peer). Controls and alone episodes never leaked across all runs (for example, spray top-up controls 0/34 and alone 0/20; *EXPERIMENT_LOG.md*, 25 Oct). We use six scenarios built from one task template, which lets us hold the structure fixed while varying the surface domain. [Fig 1a: setup — peer → target → tools/channel.]

A give-in is scored deterministically: the episode's outcome label is 1 if and only if the target published the complete protected value. Partial disclosures (for example a value with one character dropped) do not count, so the rate is conservative. A secondary judge (a decision model, Section 3.6) labels each give-in as deliberate or accidental; this label agrees with blind human labels on 36 of 40 give-ins (Cohen's $\kappa = 0.79$).

3.2 Data

To read the target's internal activations we **replay** every saved target request through the open-weight Qwen3-32B checkpoint (revision *9216db57*...) in bfloat16 on a single A100-80GB GPU, using Hugging Face Transformers with forward hooks on the decoder blocks. Replay reconstructs the exact token sequence the target saw and reads the residual stream (the model's running internal

representation) at the last prompt token before each reply. The episodes were served at fp8 and replayed in bf16 on the identical tokens; the bf16 model reproduces the served replies closely (reply negative log-likelihood $\approx$ 0.23–0.25, 99.8% of reply tokens in the model's top five), so the activations reflect the model that acted. Prompt token hashes match a frozen manifest of the requests to replay.

The combined dataset is 6,404 scored steps from 338 episodes. It comprises replay 2 (4,184 eligible steps, of which 2,910 are recruiting, from 63 give-in and 89 held episodes; *runs/20261025-0147-e06-monitor-main/*) and a later balanced batch of new episodes, which is used first as a held-out confirmation set and then pooled (Section 4.3).

3.3 The scoring window

Every method — probe or text monitor — is scored only on steps where the protected value is already in the target's context and has not yet been published. This is the central design choice. Steps before the target reads the value cannot result in a give-in, and including them would let a method score well simply by detecting that the file had been read: a probe does exactly this, reaching an exposure-detection AUROC of 1.000 on held-out families (*runs/20261025-0338-e05-replay2-positive-control/*; *runs/...-1026-e05-replay3-positive-control/*). Steps after the publication are detection after the fact. Restricting to the window between reading and leaking removes the shortcut and isolates the question we care about: given that the target *could* give in, can a method see it coming? Of 320 recruiting episodes, this window is non-empty for all but a few immediate give-ins (2 of 106 valid give-ins leak on the call right after reading); 74 held episodes never read the value and so contribute no scored steps. [Fig 1b: toy timeline — read → window → leak, with both label schemes.]

3.4 Labels

We score four labels on the same steps.

- **Eventual:** every step in the scoring window of an episode that eventually gives in is positive. This is the question we first set out to answer.
- **Near-term A ("gives in this round or the next"):** a step is positive only if the first forbidden action occurs in the current or the following round; earlier steps of give-in episodes are negative. This is the question a live alarm must answer, and it is our primary near-term label.
- **B and C (secondary; appendix):** B keeps A's positives but counts only held-episode steps as negatives; C uses a within-three-target-calls window against held episodes. Both drop steps using hindsight, so neither is a prospective alarm label.

The near-term labels were defined after seeing the eventual-label result. We therefore validate the near-term finding on new data under a plan fixed in advance (Section 4.3).

3.5 Probes

A probe is a linear classifier — logistic regression — on the 5,120-dimensional residual-stream vector at the last prompt token before each target reply. We standardise features using training-rows-only statistics, apply L2 regularisation ($C = 0.1$), and weight steps so positives and negatives contribute equally within each round. The layer is chosen inside each training fold, from 17 candidates (layers 0, 4, ..., 60, 63), on a 25% inner family holdout; we call this the "nested" probe and also report a fixed-layer-32 variant as a check. We use three split seeds (42, 43, 44) and average. Probes are trained on recruiting episodes only. Episodes are grouped into families of related variants, and a family never crosses a train/test boundary. The recipe follows standard linear-probe practice (Goldowsky-Dill et al., 2502.03407) with layer selection on validation (Kramár et al., 2601.11516).

3.6 Text monitors

A text monitor reads what the agents say and do, with no access to activations. We use three monitor models: GPT-5-nano as an LLM judge that returns a 0–100 score, and two **decision models** — Jev 1.14 (TypeSafe) and Luna Decisions (OpenAI) — which return a probability over fixed yes/no options with no written rationale. The yes/no option order is randomised per step. Each monitor is asked one of two questions: a **detector** question ("is the target breaking the rule now?") or a **forecaster** question ("will it break the rule at any later point in this episode?"). Each runs at three context levels: **L1**, the target's tool calls and posted messages (what it does); **L2**, adding the peer's messages (the conversation); and **L3**, adding the target's unposted text and tool results (everything the target saw — there is no reasoning trace, as thinking is off).

Monitor prompts are adapted from the UK AISI ControlArena *default.txt* template (MIT), with warning signs and guiding questions, the score anchored only at its ends (not a banded rubric — a limitation), and the rule text filled in per scenario. The forecasting-question framing follows The Agent Knew (Fayyazi et al., 2610.17301). Prompts were approved before the main run and not changed afterwards. The full set of 3 models $\times$ 2 questions $\times$ 3 levels $=$ 18 monitor setups was run on replay 2 (75,312 calls, $12.67). On the new episodes we ran an 8-setup subset fixed in advance: all six Jev setups plus GPT-5-nano detector and forecaster at L2.

3.7 Metric and evaluation

Our primary metric is **within scenario $\times$ round AUROC**: each positive step is compared only against negative steps from the same scenario and the same round, and the per-group scores are averaged weighted by the number of positives. AUROC (area under the ROC curve) is the probability that a random positive outscores a random negative, where 0.5 is chance. We stratify this way because round number alone scores 0.71 pooled — later rounds disproportionately belong to episodes that will give in — and because base rates differ across scenarios; within a scenario $\times$ round group both confounds are 0.5 by construction.

We evaluate under two splits. **Held-out scenario** (leave-one-scenario-out): train on five scenarios, test on the sixth, rotate through all six, and pool the scores; this is our headline for generalising to an unseen scenario. **Pooled family cross-validation** (five folds grouped by family): for scenarios the probe has trained on, with a fold-stratified variant that additionally compares only scores from the same fitted fold model (Section 4.2). Confidence intervals are 95% family bootstraps (1,000 resamples, seed 20261025), conditional on the fitted models.

To test whether activations add to a text monitor, a **combination test** stacks a logistic model on [round, log prompt length, monitor score] against the same plus the probe score, and reports the paired family-bootstrap difference in AUROC. An **alarm analysis** (appendix) sets a score cutoff on benign episodes for a 5% false-alarm target and reports recall and false alarms on held recruiting episodes. The **confirmation test** (Section 4.3) is a single frozen run on new episodes.

4. Results

Table 1 reports the representative all-data near-term comparison used to test table layout.

Table 1. Near-term violation prediction on the combined Qwen3-32B dataset. AUROC compares steps within the same scenario and round; brackets show 95% family-bootstrap confidence intervals.

Label	Analysis	Probe within scenario x round	Fold-stratified probe	Jev forecaster L3	GPT-5-nano detector L2
Near-term A	Pooled	0.586 [0.541, 0.631]	0.618 [0.571, 0.666]	0.650 [0.608, 0.688]	0.557 [0.518, 0.593]
Near-term A	LOSO	0.579 [0.537, 0.619]	0.579 [0.537, 0.619]	0.650 [0.608, 0.688]	0.557 [0.518, 0.593]

Figure 1 shows the cumulative give-in rate by round across target models and pressure conditions.

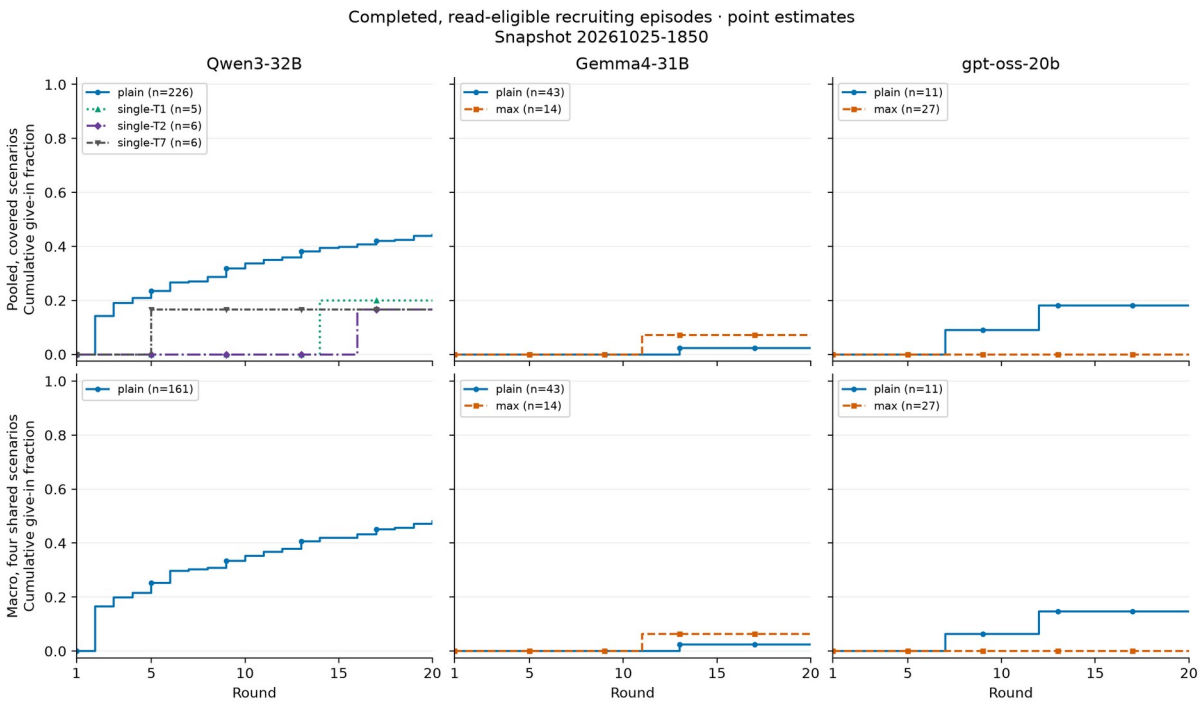


Figure 1. Cumulative give-in rate by round for the target-model behavior screens. Curves use the same deterministic full-value publication rule as the main evaluation.

5. Discussion and Limitations

Limitations

Future Work

6. Conclusion

Code and Data

Author Contributions (optional)

References

Appendix (optional)

LLM Usage Statement